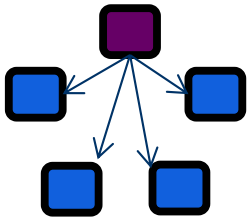
CompSci 390-03-26S

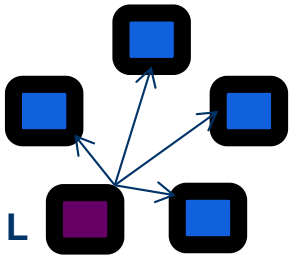
Distributed and Networked Systems

BASE: Eventual Consistency

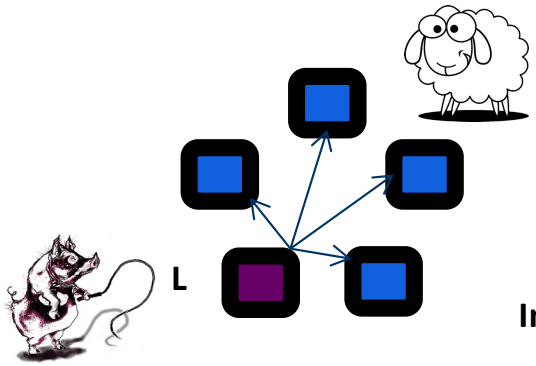


State Machine Replication (SMR/RSM)

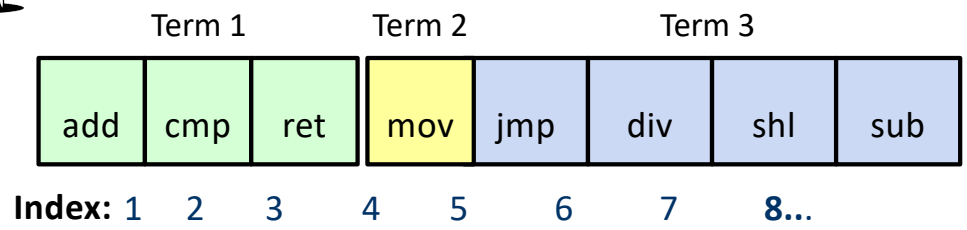
- N replicas: N = 2f+1 to mask up to f faults—highly available, "always on".
- Replicas run the same app program P with the same API. It doesn't matter what P is (reusable!), but P must be **deterministic**.
  - P is a **state machine**: function {state, action}→state
- Each replica keeps its own copy of internal app state for P.
- **Consensus**: replicas agree on a sequence (or log) of actions to apply.
  - Ordered list of (requests, commands, ops). **Linearizable**.
- Then the replicas converge. E.g., by Raft:
  - S/elect stable leader
  - Leader handles client RPCs
  - Others follow along as **backups**



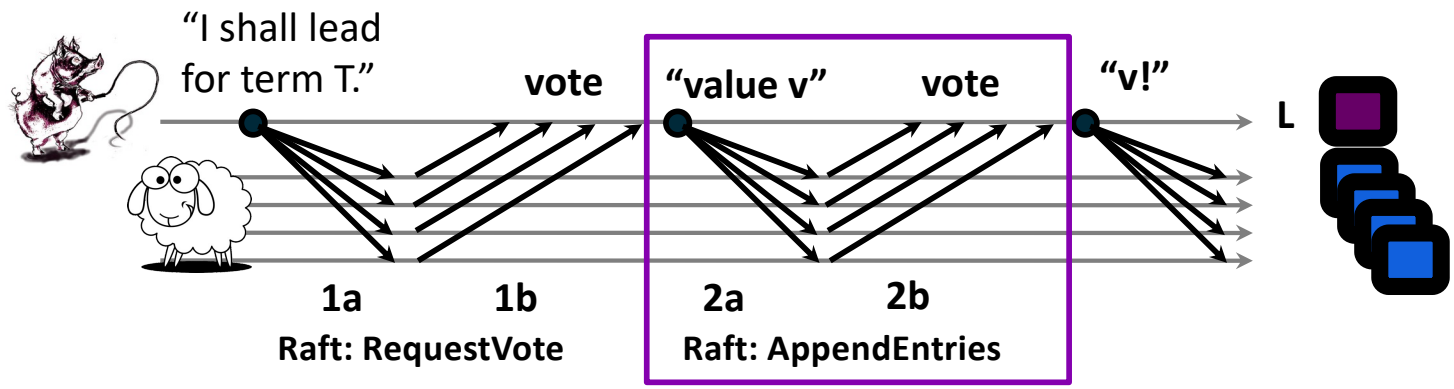
SMR consensus: in pictures



A happy group of 5 replicas.

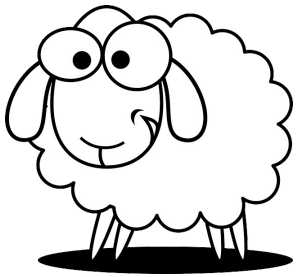


Each has a copy of a log of (term, op) pairs @ each index.
Term (ballot, view); **index** (slot)**; op** (command, action).



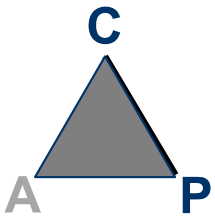
Consensus oversimplified

1. If leader falters try to take over.
2. Become leader by majority vote/assent.
3. Discover and affirm history.
4. Append new log entries in order; notify others if majority accepts.



1. Adopt leader.
2. Tell it your history.
3. Accept whatever the leader says.
4. Write it all down.
5. If a new leader appears, goto step 1.

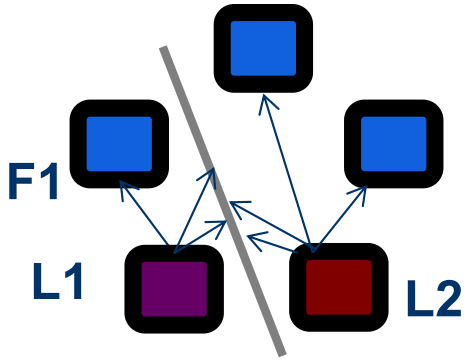
Consensus: Quorum rule



In order to serve and commit, a leader L1 must continually receive votes from a **majority** or **quorum**: f+1 of the full 2f+1 group.

- **Safe** (C). In a partition scenario, at most one subgroup can serve requests.
- But not **live** (A). If a majority of replicas fail, it might be safe for the survivors to serve. But they **must not** because they cannot rule out a partition.

L1 and F1 do not have a quorum. So they must not serve clients → no **A**vailability if the majority had merely failed.

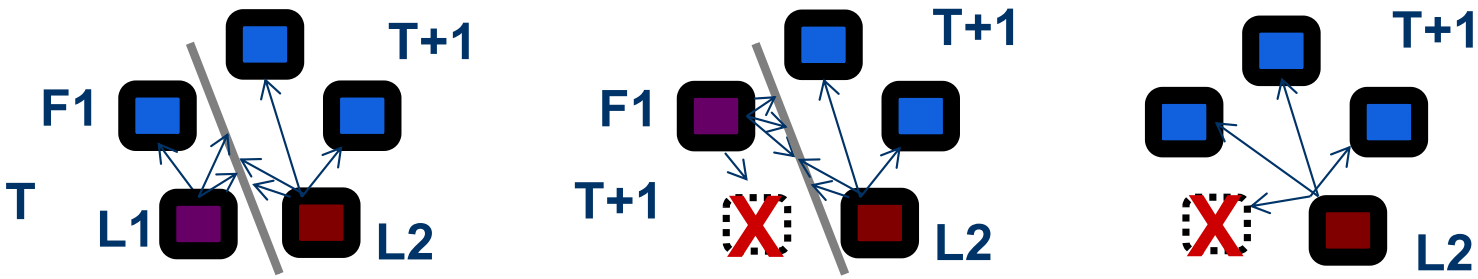


L2 can gain a quorum from the other followers, so its side of the partition can make progress.

At most one side of a partition makes progress because **disjoint majorities cannot exist**.

Never two kings at once

- Multiple replicas may try to lead with a given term T, but at most one can win a quorum to lead in T; the loser(s) **never** create entries with term T.
- Therefore, if two replicas have divergent entries at log index i, then the entries to keep (e.g., committed by new leader) **always** have a higher term.



L1 leads in term T, then: partition! And then L1 fails! F1 and L2 declare candidacy with term T+1. But only L2 wins a quorum. F1's candidacy is doomed: it never becomes leader, and so never accepts a client request, and never creates a log entry with term T+1. When the partition heals, F1 hears and follows L2 in T+1.

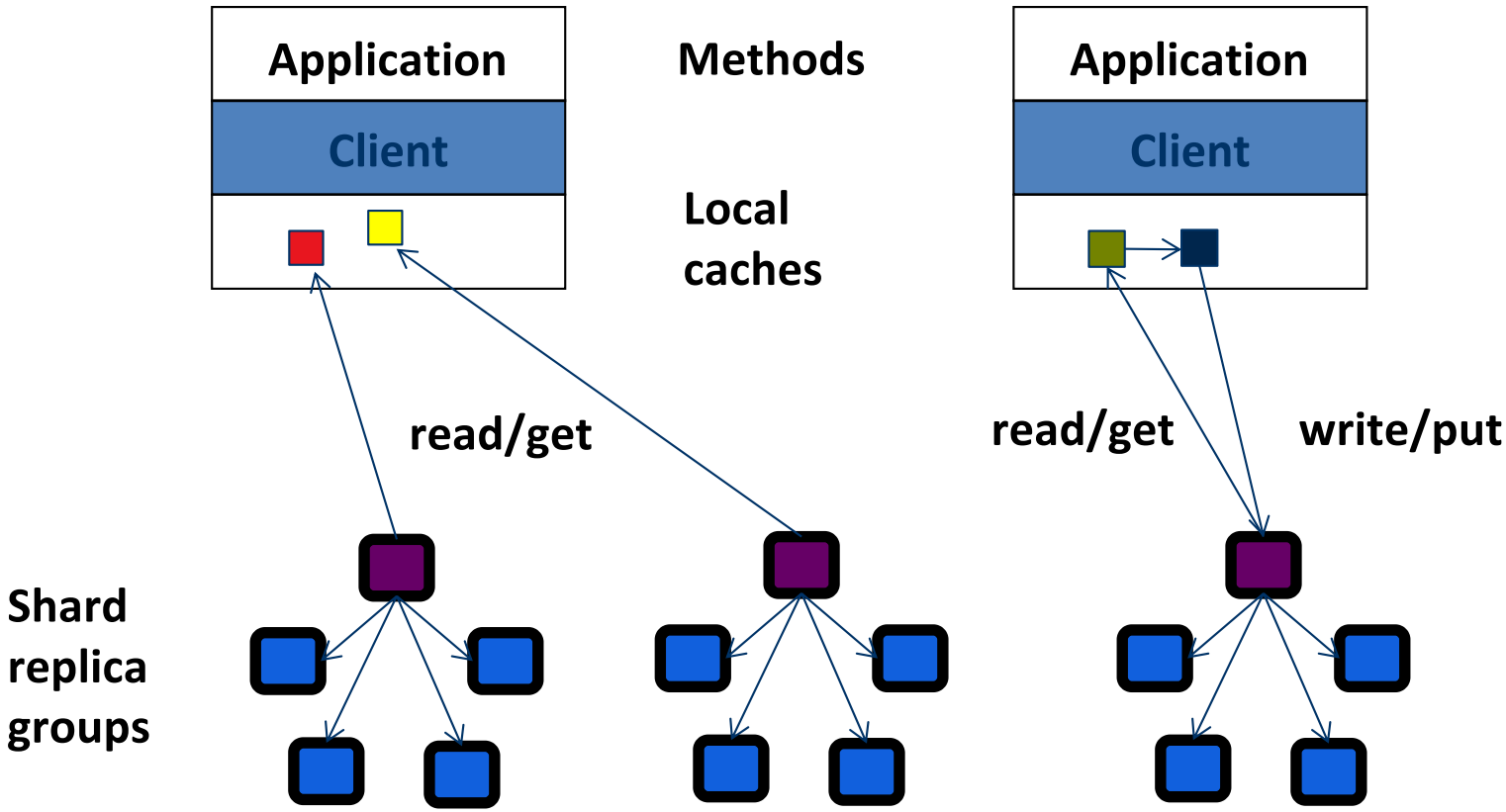
Costs of consensus

- **Scale?**
- **Latency?**
- **Availability?**

Costs of consensus

- **Scale?** No benefit! Must add sharding.
- **Latency?** Bounded by slowest server in quorum.
  - High **tail latency** especially for geo-replicated services
- **Availability?** Unbounded hanging states.

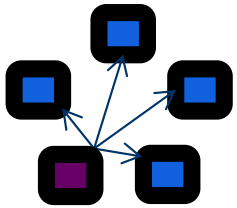
Sharding and replication together

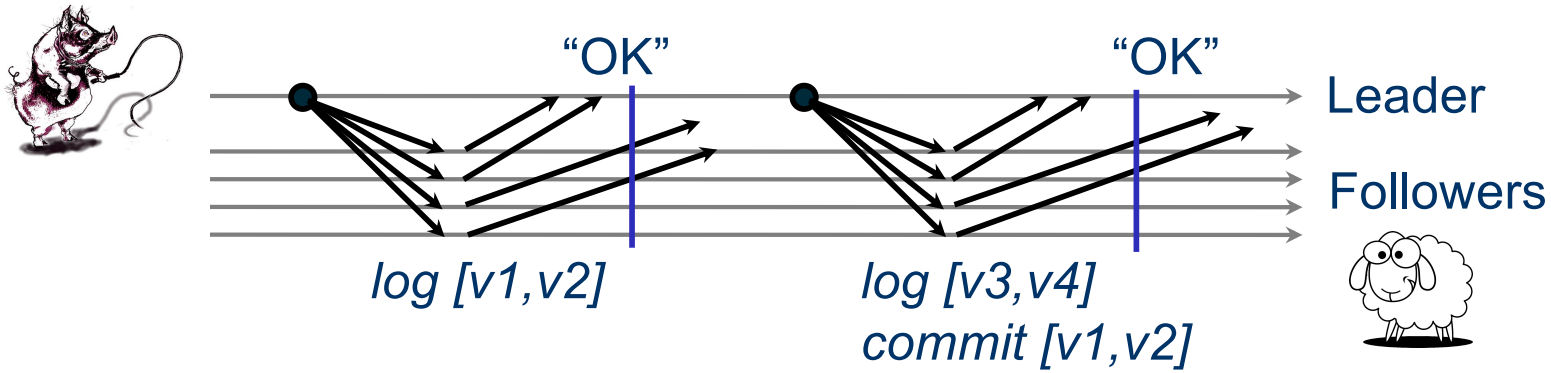


Serving and committing entries

AppendEntries @i
entries: [v1,v2]
committed=i

AppendEntries @i+2
entries: [v3,v4]
committed=i+2



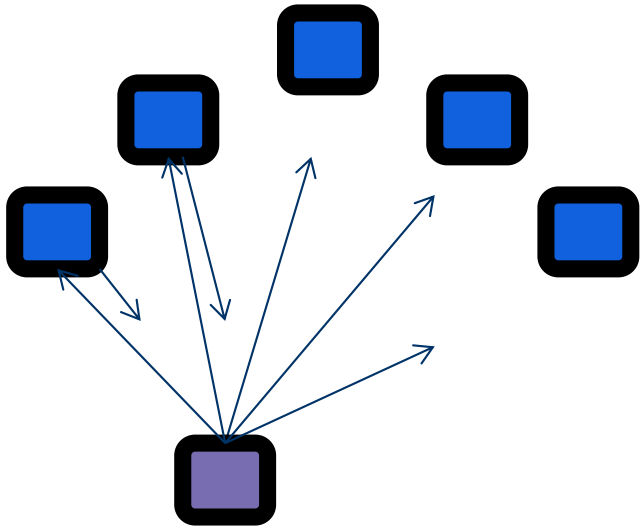


Note that **log** here means to add a record to the replicated log (in memory) and **commit** means the committed entry will never go back.

What if replicas lag? **When is it safe to commit?**

Server latency (R) varies

- Network/light delays
- Server load imbalance
- Resharding, Failover
- Scheduler "crosstalk"
- Energy throttling
- Queues in network
- Background processes
- Garbage collection
- Other maintenance



Message latency and per-server R follow probability distributions. Consensus latency is bounded by the slowest server in the quorum (median server).

Amazon on tail latency

Focus on tail latency:

A common approach in the industry for forming a performance-oriented SLA is to describe it using average, median and expected variance. At Amazon we have found that these metrics are not good enough…

In this paper there are many references to this 99.9th percentile of distributions, which reflects Amazon engineers' relentless focus on performance from the perspective of the customers' experience. Many papers report on averages, so these are included where it makes sense for comparison purposes. Nevertheless, Amazon's engineering and optimization efforts are not focused on averages.

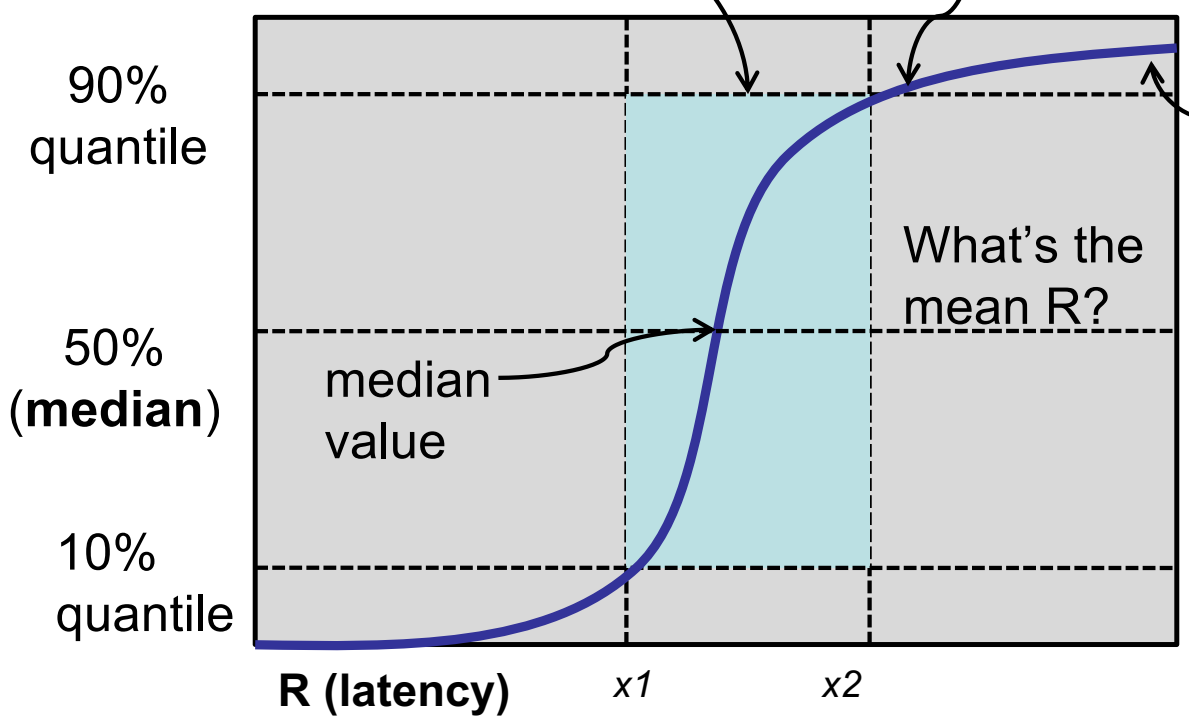
powered by amazon web services™

Text snippet from Dynamo: Amazon's Highly Available Key-value Store (SOSP 2007)

Mind the quantiles! (CDF)

80% of the requests (90-10) have response time *R* with *x1* < *R* < *x2*.

"**Tail**" of 10% of requests with response time *R* > *x2*.

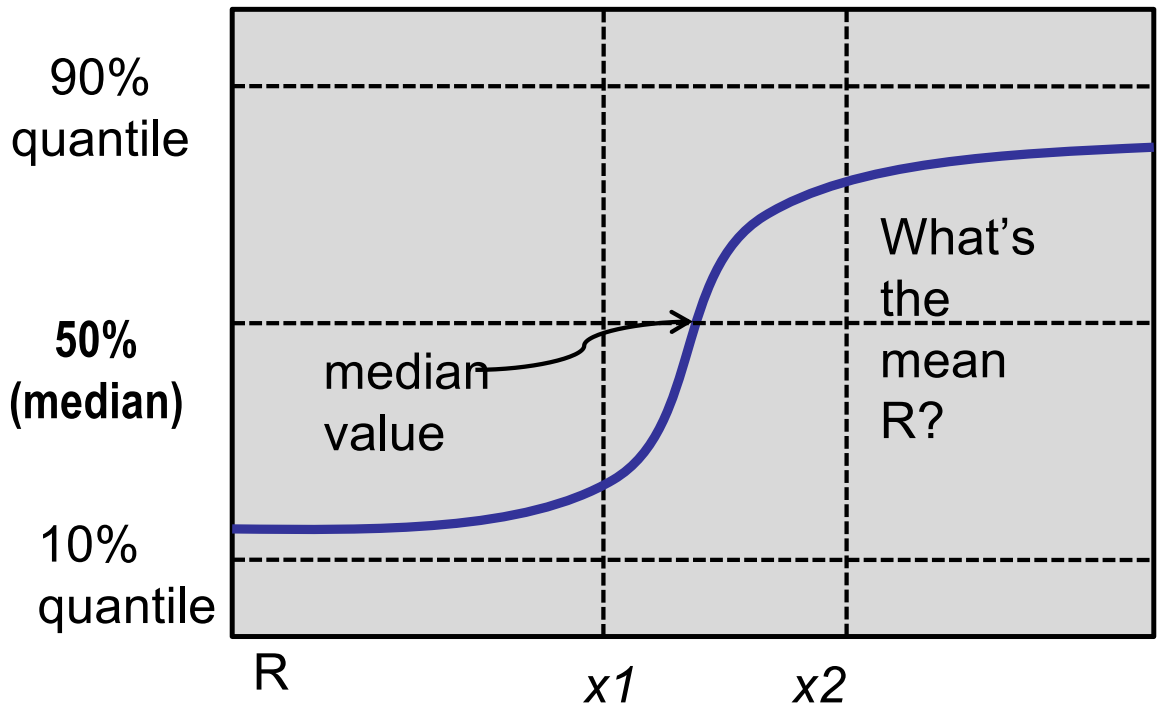


A few requests have very long response times.

<u>Understand</u> how/why the mean (average) response time can be misleading.

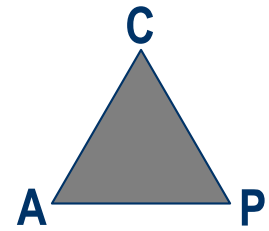
Understanding latency profiles

Better? Worse?

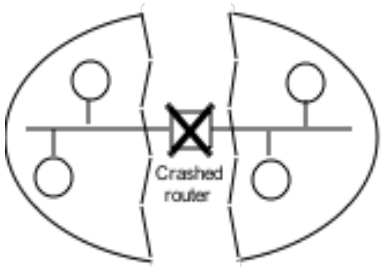


CAP: safety vs. liveness

- **Safety**: nothing bad happens: e.g., **consistency**.
  - Single System Image (1SR): read X returns "last" write.
- **Liveness**: something good eventually happens: **availability**.
- In an asynchronous network with failures and **partitions**
- Safe (consistent) consensus may occur recognizably, but liveness (availability) is not guaranteed.



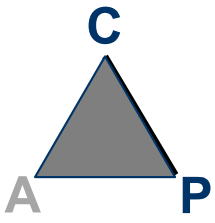
Choose two.



Network partition

FLP popularized as Brewer's Conjecture or CAP trilemma. **No you can't have it all**: **C**onsistency + **A**vailability in presence of **P**artitions.

Consensus (Raft) is CP on the CAP triangle



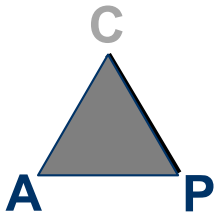
- Single timeline of operations (e.g., writes).
  - Time is a sequence of terms/views.
  - At most one viable leader per term.
  - Leader decides a serial order of requests in the term.
- The server for every client request knows all previous writes.
  - Majority quorum needed to serve requests (as leader).
  - Majority quorum needed to commit a write.
  - Any two majorities must intersect. Therefore...
- Reads serialized with writes→service is **linearizable**.
  - Assuming consistent caches (e.g., Zooky with session leases).

An easier way?

- What if we just relax?
- Or take a more flexible view of quorums (quora)?
  - Complete writes without a majority
  - Prioritize reads, e.g., read from any
- Or: allow any server to serve requests (no strong leader)?
- What is the impact on applications? Do they still work?
- How much consistency can we afford to lose? What weaker models are useful? How to quantify consistency?
- What if we prioritize liveness (availability)?

AP: Extreme Availability

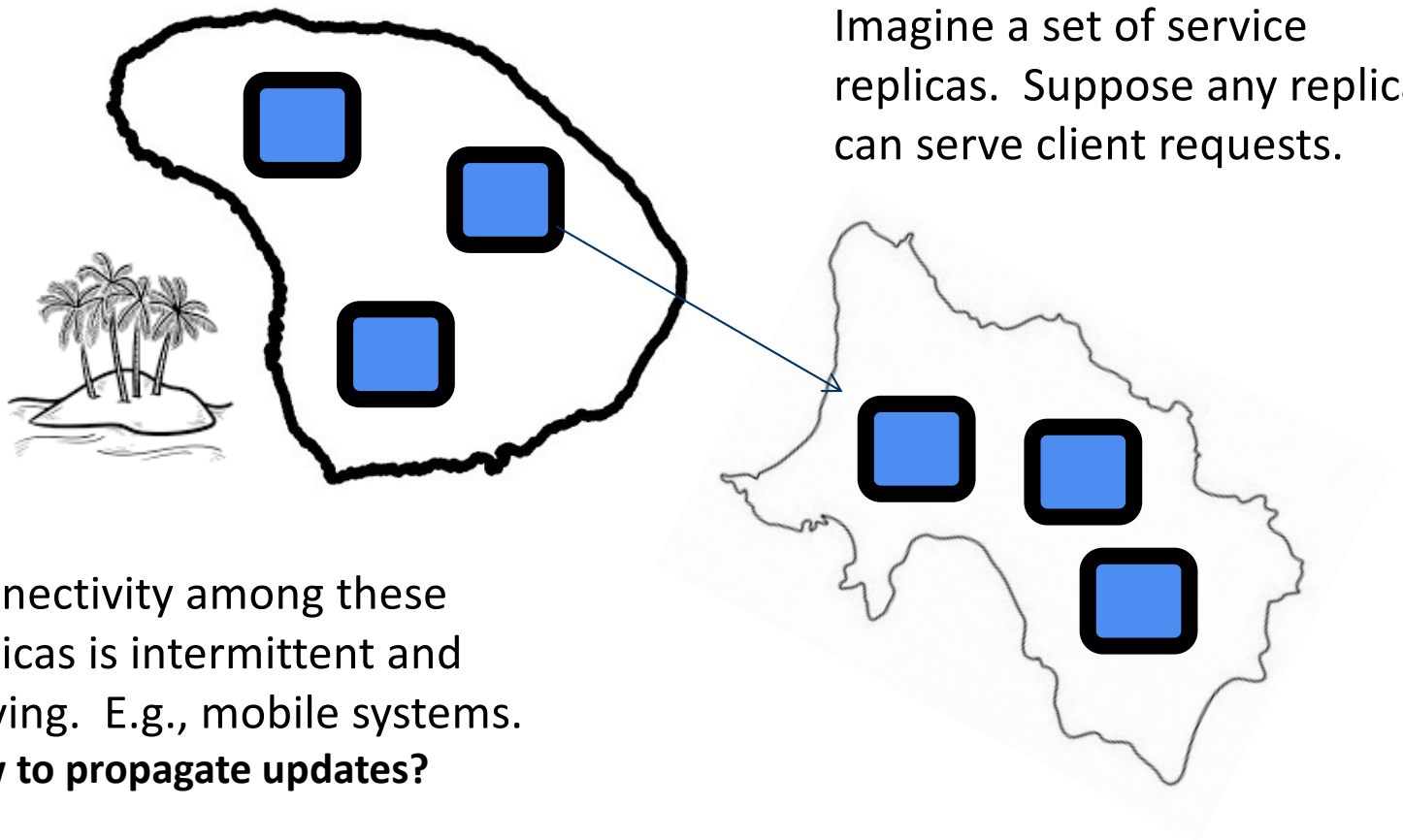
- What does it mean to design for availability when network connectivity is intermittent? **AP**
- **CAP/FLP** says we must give up 1SR ("single system image"): replicas may have different states. **No C**.
- **How can we build systems that work for applications in this world?** We need some weaker notion of consistency that still works.
  - **Eventual consistency**
  - **Causal consistency**: logical clocks & vector clocks
  - How to fork/merge concurrent timelines?
  - Style called BASE, CALM or ACID 2.0



Some history: the idea of BASE

- **B**asically **A**vailable **S**oft State **E**ventually Consistent
  - Another great contrived acronym from Berkeley [Brewer]
- "The opposite of ACID" for the early mega-services era
- **Focus**: design for scale and availability
  - Shard and replicate
  - Serve requests where you get them
  - Give the best answer you can fast
  - Keep going
- **In retrospect**: transition from peer-to-peer era (DHT) to first-gen data center stacks based on loose key-value stores (1999-2012), including Amazon **Dynamo** (2007).

Extreme AP: replicas on island travel



Imagine a set of service replicas. Suppose any replica can serve client requests.

Connectivity among these replicas is intermittent and varying. E.g., mobile systems.
How to propagate updates?

Dynamo:

Amazon's Highly Available Key-value Store

SOSP 2007

powered by amazon web services™

basho riak

LinkedIn

Project Voldemort

Apache, DataStax

Cassandra

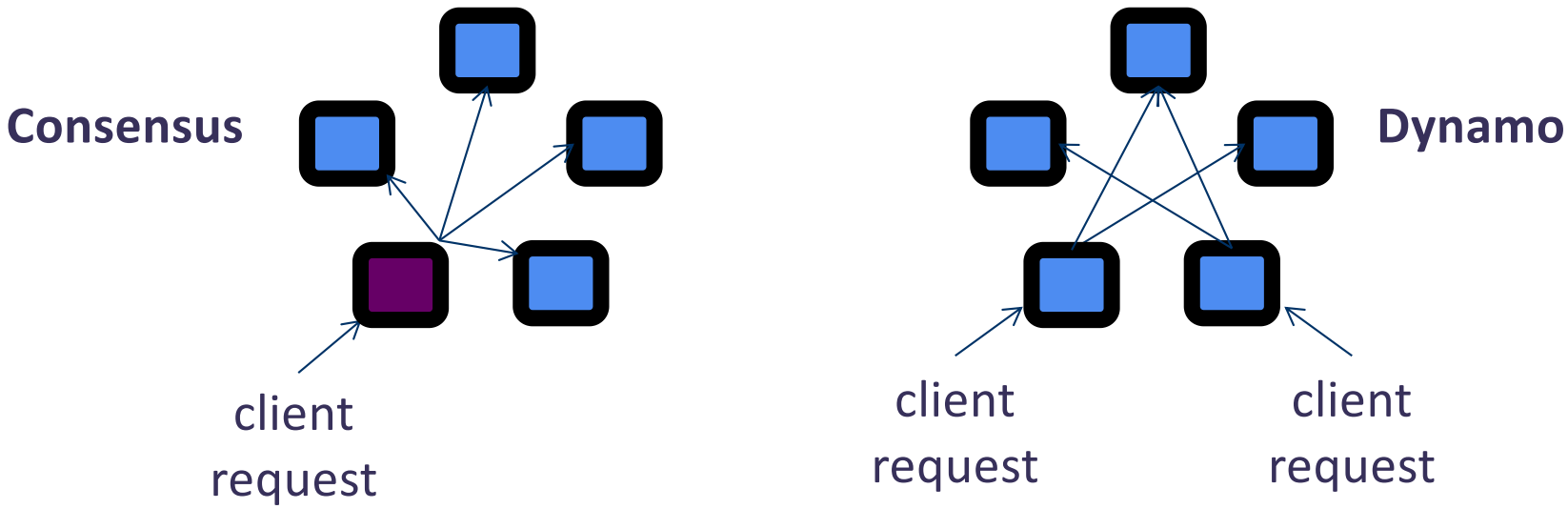
[Peter Bailis – Basochats talk]

Dynamo: replication

- Dynamo is a **highly available** KV store.
- Dynamo uses replication to mask failures.
- Each (key, value) pair is stored on **N** servers/replicas.
- Sharding based on **consistent hashing:** each shard has a different replica set. (Replica sets may overlap.)
- Dynamo uses a "sloppy" variant of consensus replication called RWN quorum replication.
- RWN gives better latency, much better tail latency, and an assurance of availability (AP).

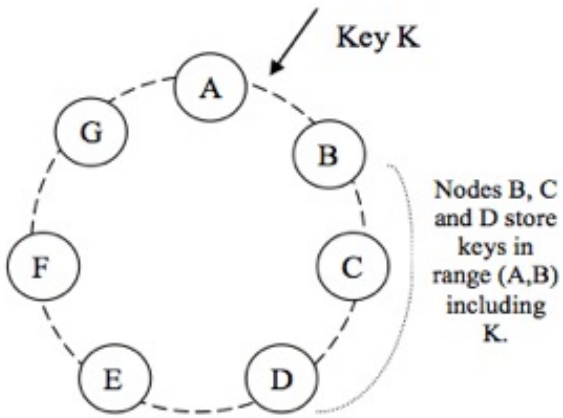
Dynamo vs. consensus

- Dynamo: no designated primary/leader!
- Any replica may serve as **coordinator** for a write.
  - Coordinator multicasts to all peers, but may complete without majority.



Dynamo: many kings!

- A Dynamo shard has no designated primary.
- No leader election, no mutual exclusion
- Allows more concurrency, lower latency
- Anyone can take charge! Anarchy?



Although it is desirable always to have [a leader replica] coordinate the writes thereby serializing all writes at a single location, this approach has led to uneven load distribution resulting in **SLA violations**. This is because the request load is not uniformly distributed across objects. To counter this, any [replica] is allowed to coordinate the writes. In particular, since each write usually follows a read operation, the **coordinator for a write is chosen to be the node that replied fastest** to the previous read...

RWN quorum consistency

- **Read and write operations issue to all N replicas**. Coordinator or client may issue them—multiple coordinators OK.
- An op completes only after some minimum number (a **quorum**) of replicas ack/accept the op: **W** or **R**.
  - Complete a write after **W** replicas respond, complete a read after **R** replicas respond (pick response with "most recent" value).
- Set the quorum size so that any **read quorum** (of size **R**) is guaranteed to overlap with any **write quorum** (of size **W**).
  - Surely any read "sees" the value of the "latest" write.
- Ensures strong (single copy) consistency, but it still must deny service if "too many" replicas fail or become unreachable.

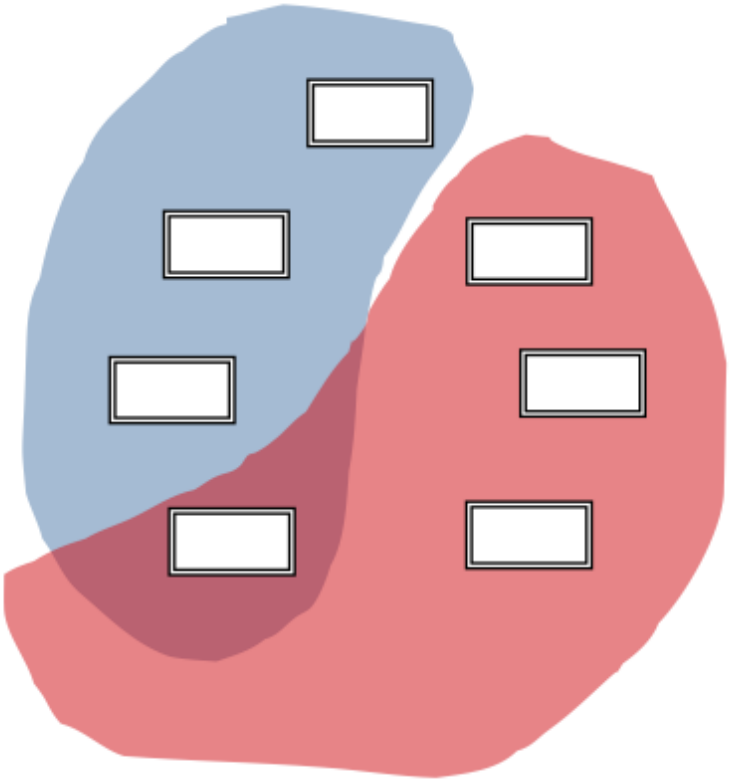
RWN weighted quorum

N=7

R=4

W=4

???



[Keith Marzullo]

RWN weighted quorum

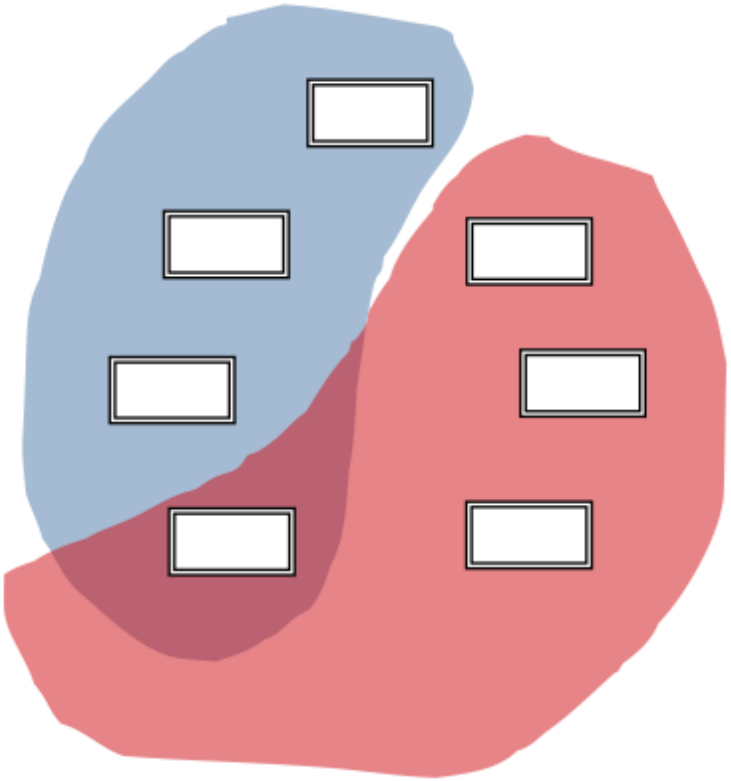
N=7

R=4

W=4 ✅

R+W > N

W>N/2



Any write quorum must intersect every other quorum.

[Keith Marzullo]

Dynamo: “sloppy quorum”

Dynamo allows configurations that set R and W lower than a full quorum.

- Set RWN parameters to meet your needs.
- R+W < N+1 → trade off consistency for latency and availability.

Due to these factors, it is possible that a read quorum on key K does not intersect the previous write quorum for K.

- You might read a value other than the last value written.
- Completed writes might “conflict”.

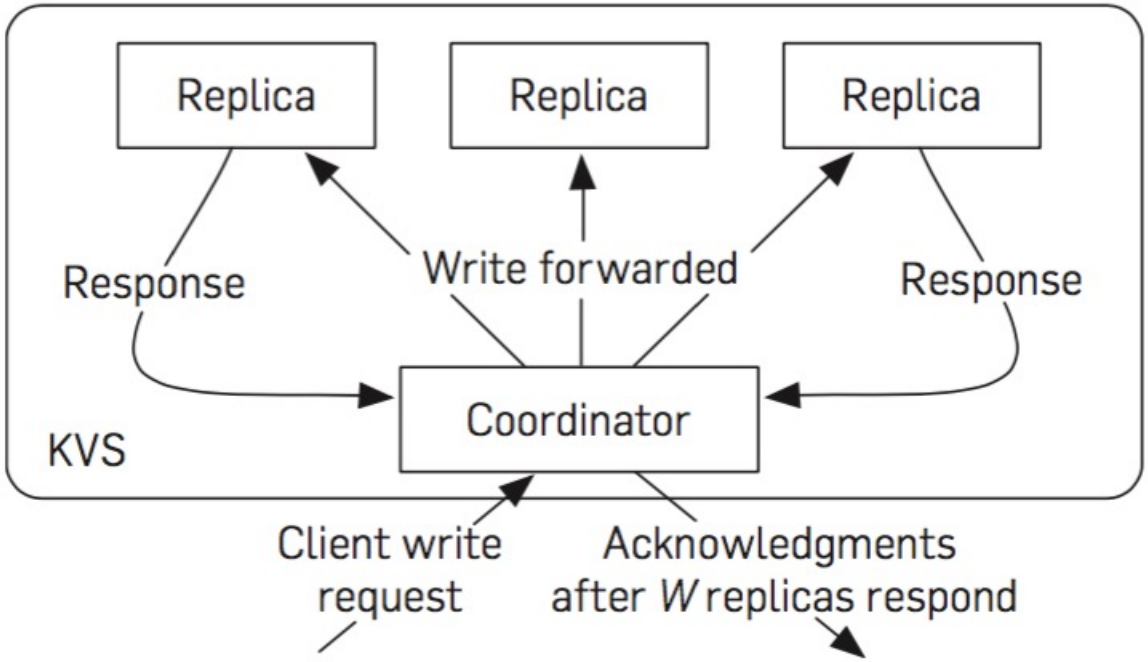
Sloppy quorum

Read and write operations involve the first N healthy nodes in the preference list, skipping over those that are down or inaccessible. When all nodes are healthy, the top N nodes in a key's preference list are accessed. When there are node failures or network partitions, nodes that are lower ranked in the preference list are accessed….

If Dynamo used a traditional quorum approach it would be unavailable during server failures and network partitions, and would have reduced durability even under the simplest of failure conditions. To remedy this it does not enforce strict quorum membership and instead it uses a "sloppy quorum"; all read and write operations are performed on the first N *healthy* nodes from the preference list, which may not always be the first N nodes encountered while walking the consistent hashing ring.

A write

Figure 1. Diagram of control flow for client write to Dynamo-style quorum (*N* = 3, *W* = 2). A coordinator server handles the client write and sends it to all *N* replicas. The write call returns after the coordinator receives *W* acknowledgments.



[PBS Bailis et. al. CACM 2014]

RWN replication

- What if R+W < N+1?
- **Good**: reads and/or writes don't have to wait for a full quorum of replicas to respond → lower latency.
  - And *much* lower tail latency.
- **Good**: better availability in failure scenarios.
- **Bad**: reads **may** return stale data → eventual consistency
- **Bad**: write conflicts **may** occur
- **Bad**: replicas **may** diverge temporarily.

Diverge and fix it later?

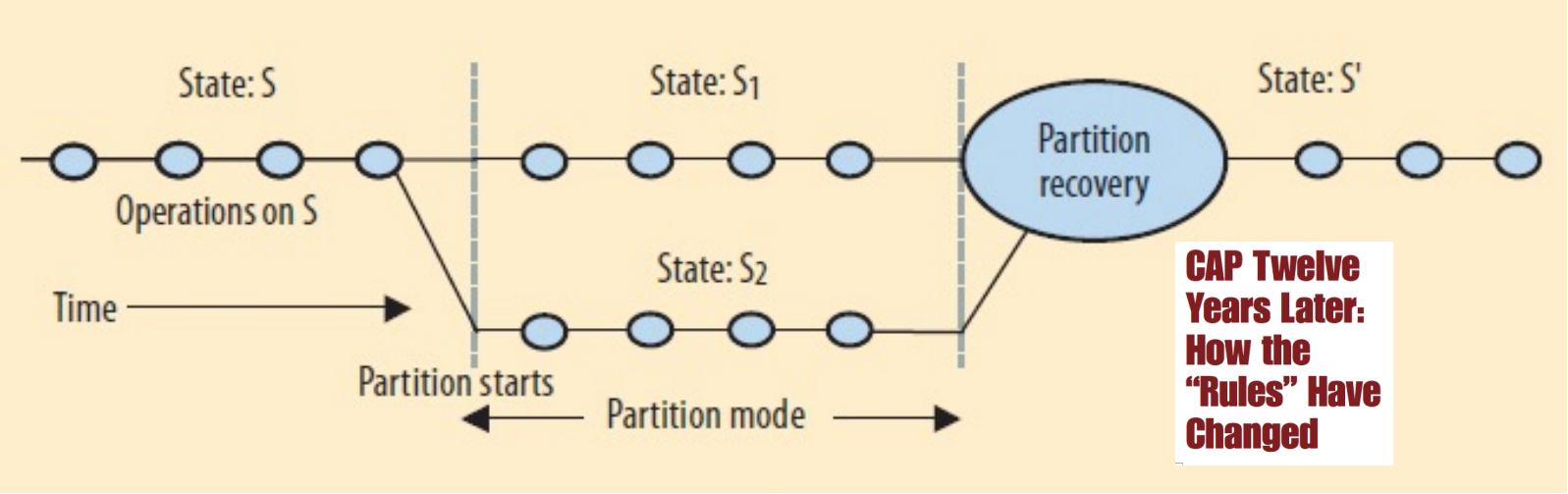


Figure 1. The state starts out consistent and remains so until a partition starts. To stay available, both sides enter **partition mode** and continue to execute operations, creating concurrent states S1 and S2, which are inconsistent. When the partition ends, the truth becomes clear and partition recovery starts. During recovery, the system merges **S1** and S2 into a consistent state S' and compensates for any mistakes made during the partition.

Image borrowed from Eric Brewer *CAP 12 Years Later (2012)*

Dynamo: Partition recovery

How to "repair consistency" when collisions occur?

How to detect conflicting writes? **Vector clocks**

How to reintegrate conflicting writes? **Kick to app**

For given R, W, N:

- How to quantify the tradeoffs and likelihood of stale data?
- How to quantify the latency benefits? **PBS**
- Requires understanding the workload and network: distributions of requests and network latencies in time and space.
- [PBS and vector clocks to be discussed later.]

Dynamo: multi-version KV store

- Each value has a **version stamp**, also called **context**.
- Why does **Put** take a stamp for the last version read?
- Why can **Get** return multiple values?
- Under what circumstances can this occur?
- If there are multiple values, the app is responsible for reconciling them. How?

```
Get(key) → {[version, value]*}
Put(key, version/context, value) → success
```

Dynamo's "killer app": shopping carts

For a number of Amazon services, rejecting customer updates could result in a poor customer experience. For instance, the **shopping cart** service must allow customers to add and remove items from their shopping cart even amidst network and server failures. This requirement forces us to push the complexity of conflict resolution to the reads in order to ensure that writes are never rejected.

Shopping carts are sets: just take the union

For example, the shopping cart application requires that an "*Add to Cart*" operation can never be forgotten or rejected. If the most recent state of the cart is unavailable, and a user makes changes to an older version of the cart, that change is still meaningful and should be preserved. But at the same time it shouldn't supersede the currently unavailable state of the cart, which itself may contain changes that should be preserved. …When…the latest version is not available, the item is added to (or removed from) the older version and the divergent versions are reconciled later. …

For example, the application that maintains customer shopping carts can choose to "merge" the conflicting versions and return a single unified shopping cart…Using this reconciliation mechanism, an "add to cart" operation is never lost. **However, deleted items can resurface**.

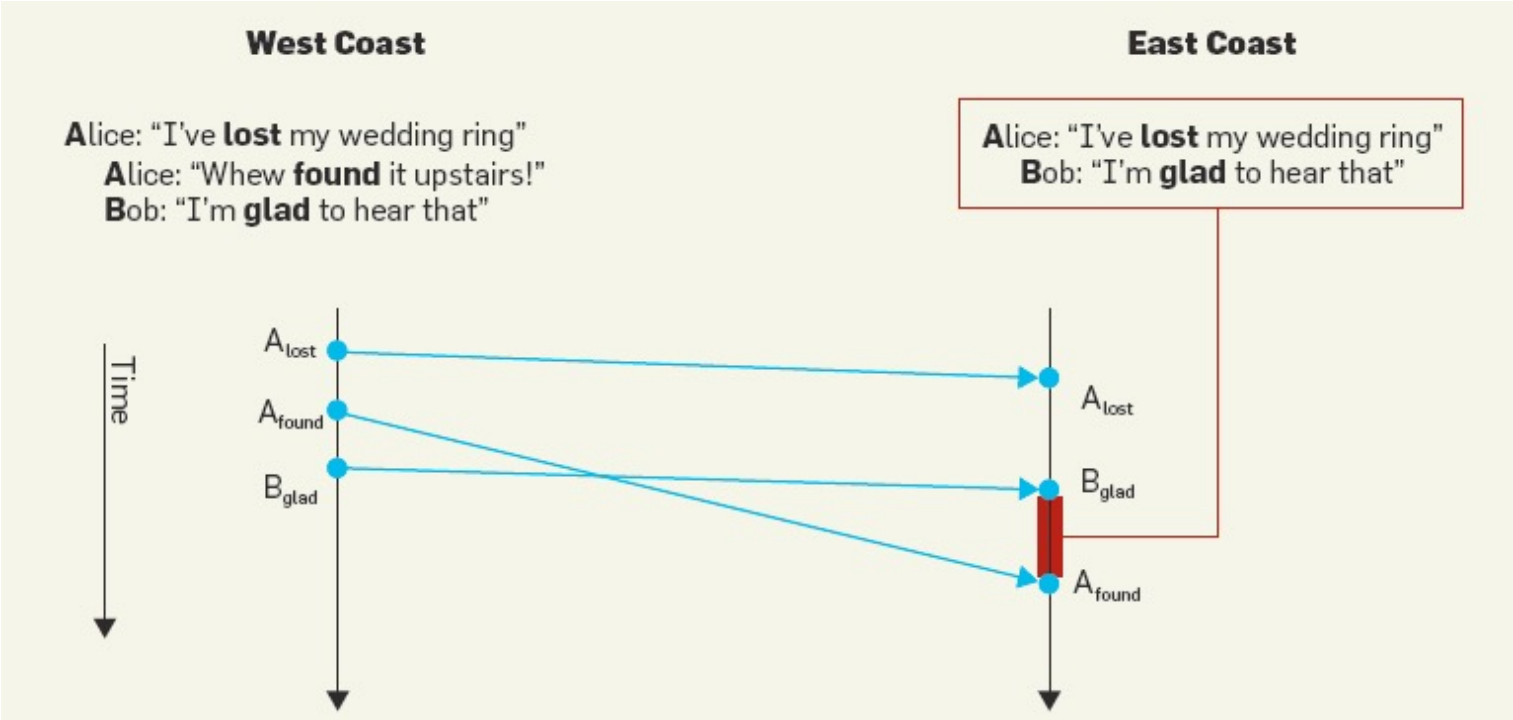
CALM and CRDTs

- A shopping cart is a **set**, an example of a conflict-free replicated data type (CRDT) with a lattice of states.
- Set union is commutative, associative, idempotent: it doesn't matter what order we add items: same output.
- Union is a **monotonic** operator: a program P is monotonic if for any input sets S,T where $S \subseteq T$, $P(S) \subseteq P(T)$.
  - "Never take it back."
- But what about deleting items? …

Keeping CALM: points

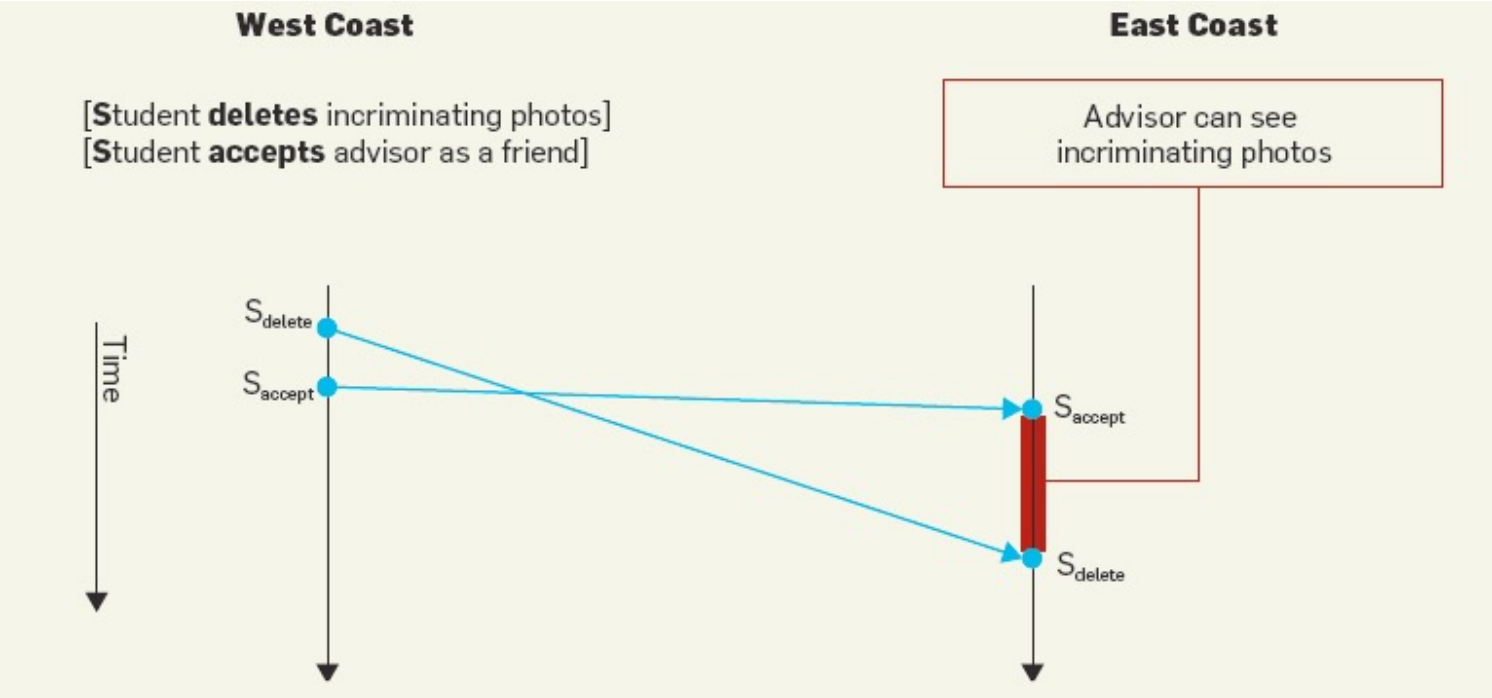
- **Theorem**. Consistency As Logical Monotonicity (CALM). A program has a consistent, coordination-free distributed implementation if and only if it is monotonic.
- **Observation**. Coordination-freeness is equivalent to availability under partition.
- CAP is a negative result over *arbitrary* programs.
- CALM is a positive result: "CALM circumscribes the class of problems for which all three of the CAP properties can indeed be achieved simultaneously."
- How many applications are CALM?

Causal inconsistency: example



[COPS]

Causal inconsistency: example



[COPS]